UNIVERSITÀ DI TRENTO

Deliveroo.js

Autonomous Software Agents

a.y. 2025/2026

Davide Donà - 264467 Andrea Blushi - 264468

June 2026

1 BDI

1.1 Belief Revision

1.2 Intention Revision

1.3 Game Strategy

Some maps of the Deliveroo.js environment present

2 LLM Based Agent

2.1 Request

2.2 Adaptive Behaviour

3 Coordination Strategies

4 PDDL

A difficult aspect of developing an autonomous agent for the Deliveroo.js environment is the fact that some maps present movable objects with specific constraints. In particular, two maps contain crates that can be moved by the agents and may block their paths. We decided to model the problem of the agents moving in such an environment through the use of PDDL (Planning Domain Definition Language).

4.1 Domain Overview

The PDDL domain acts as the foundational physics engine for the agent's internal model of the environment.

4.2 Planning With PDDL

4.3 A Smart Solution - Problem Restatement & Caching

Since the PDDL planner introduced a lot of overhead that was not acceptable for the reactive nature of the environment, we decided to implement a smarter solution. The idea is to restate the problem in a way that allows us to cache the plans for later use.

In particular, instead of invoking the planner to compute a global solution from the agent's current position to the final goal, we decomposed the plan into 3 steps:

- Move from the current position to the first crate in the path to the goal;
- Move through the crate section;
- Move from the last crate to the goal.

This way, while the first and third steps can be solved with a simple pathfinding algorithm (we restrict the PDDL planner to only solve the second step, which is the most complex one. This allowed us to cache the generated plans for the second step. In fact, since the original problem formulation was from point to point, simply moving either the start or the goal by one tile would have resulted in a cache miss, even though the PDDL plan would have been exactly the same.

By decomposing the problem into three steps, we can have a vastly higher cache hit rate. The localized PDDL problem now only depends on the relative arrangement of the crates and the agent's immediate entry and exit points for that specific cluster, rather than the absolute global coordinates. As a result, the exact same crate-manipulation plan can be reused whenever the agent encounters a structurally identical sub-problem, drastically reducing computational overhead and ensuring the agent remains highly reactive.